

CodeAlpha Project Management Tool - Complete Learning Guide

This document is designed to help you understand the entire project from an interview perspective. The goal is not to memorize code. The goal is to understand why each file exists, how frontend and backend communicate, how MongoDB stores data, and how the application works as a complete system.

1. Project Overview

Project Name: Project Management Tool

Features implemented:

- Create Project
- View Projects
- Delete Project
- Update Project Status

Tech Stack:

Frontend: React + Axios

Backend: Node.js + Express.js

Database: MongoDB + Mongoose

Flow:

User -> React Frontend -> Express API -> MongoDB -> Response -> React UI

2. Backend Architecture

server.js starts the Express server and connects MongoDB.

Routes:

projectRoutes.js receives requests.

Controllers:

projectController.js contains business logic.

Models:

Project.js defines the MongoDB schema.

Database:

MongoDB stores project data permanently.

3. What Each Backend File Does

Project.js:

Defines fields:

title, description, status.

projectController.js:

createProject() -> inserts data.

getProjects() -> fetches all projects.

deleteProject() -> removes project.

updateProjectStatus() -> updates status.

projectRoutes.js:

POST /api/projects

GET /api/projects

DELETE /api/projects/:id

PUT /api/projects/:id

4. Frontend Architecture

Dashboard.jsx:

Main page.

Loads projects and handles CRUD operations.

ProjectForm.jsx:

Collects user input.

ProjectList.jsx:

Displays projects and action buttons.

projectService.jsx:

Contains Axios API calls.

Acts as bridge between React and backend.

5. Full Request Flow

When Add Project is clicked:

1. User enters title and description.
2. ProjectForm calls onAddProject().
3. Dashboard calls createProject().
4. Axios sends POST request.
5. Backend route receives request.
6. Controller creates MongoDB record.
7. MongoDB returns saved project.
8. Frontend refreshes project list.

6. MongoDB and Mongoose Concepts

MongoDB is a NoSQL database.

Mongoose helps Node.js interact with MongoDB.

Schema:

Defines structure of data.

Model:

Used to perform CRUD operations.

Examples:

Project.create()

Project.find()

Project.findByIdAndDelete()

Project.findByIdAndUpdate()

7. Interview Questions You May Get

Q: Why did you use React?

A: Component-based architecture and state management.

Q: Why Express?

A: Lightweight backend framework for APIs.

Q: Why MongoDB?

A: Flexible NoSQL database suitable for JSON-like documents.

Q: What is Axios?

A: Library used to send HTTP requests from frontend.

Q: Explain CRUD.

A:

Create -> POST

Read -> GET

Update -> PUT

Delete -> DELETE

Q: What is a REST API?

A: Standard way for frontend and backend to communicate using HTTP methods.

8. Biggest Lessons Learned

- How frontend talks to backend.
- How APIs are created.
- How MongoDB stores data.
- How React state updates UI.
- How debugging works.
- How routes, controllers, models and services connect together.
- How a full-stack application is structured.

9. Memory Map

Remember this chain:

User



React Component



Service (Axios)



Express Route



Controller



Mongoose Model



MongoDB

Response comes back through the same path in reverse.

If you remember this chain, you understand the project.